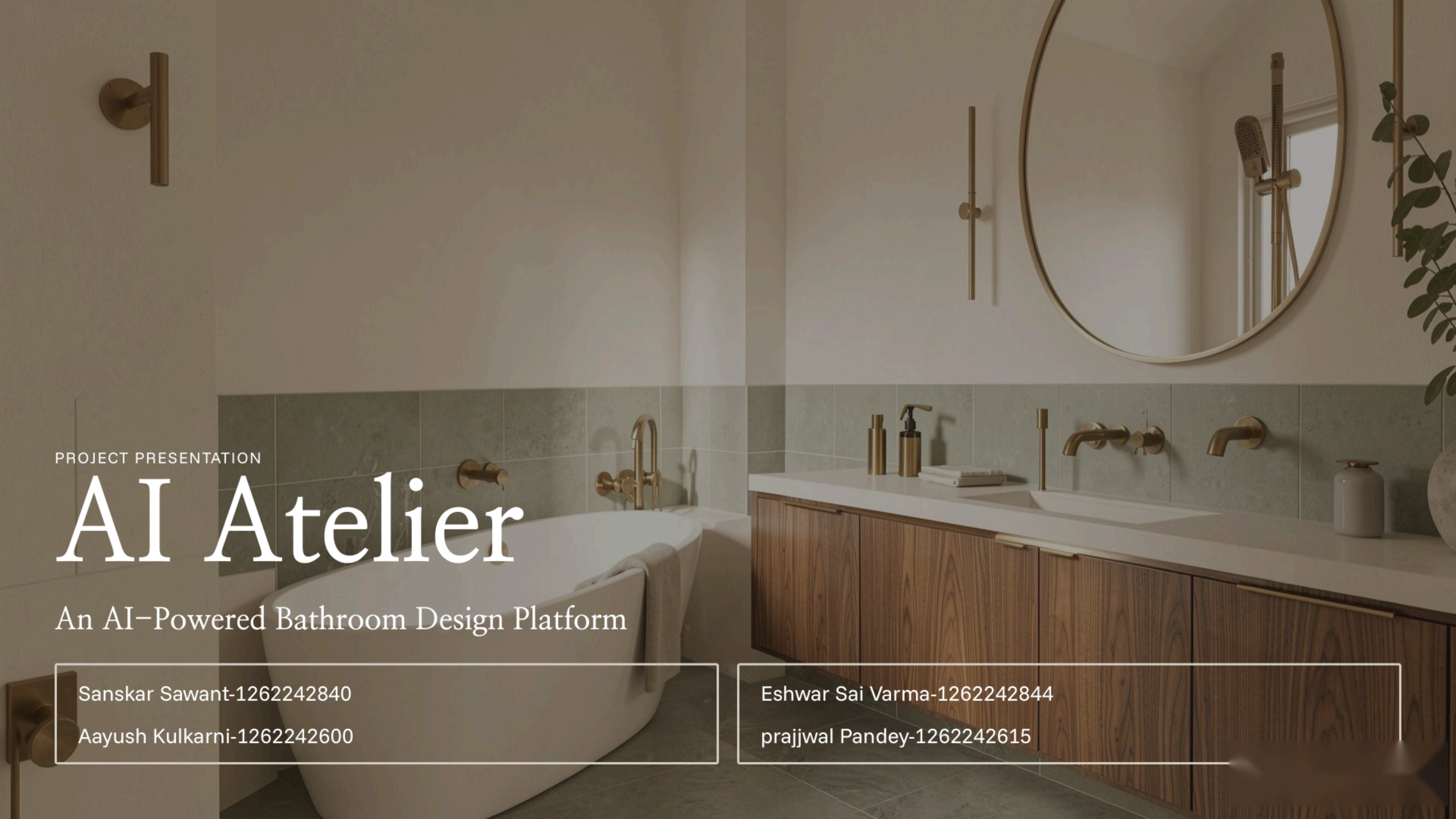


PROJECT PRESENTATION

AI Atelier

An AI-Powered Bathroom Design Platform

Sanskar Sawant-1262242840

Aayush Kulkarni-1262242600

Eshwar Sai Varma-1262242844

prajjwal Pandey-1262242615

THE PRODUCT IDEA

From room details to a bathroom concept

AI Atelier is intended to help users design or redesign a bathroom by combining room information, style preferences, budget, product recommendations, layout data, and AI-assisted visualization.

Input: room details, budget, style, or an existing bathroom photo

Output: recommended products, layout information, and an image-generation result when configuration and external services work



INTENDED PROBLEM AREAS

Renovation planning is hard before the first decision

Visualize the finished room

It is difficult to visualize the finished room before renovation.

Match style and budget

Products need to match both the style and the available budget.

Explore alternatives

People need alternatives when the first design direction does not fit.

Redesign what exists

Existing bathrooms may need redesigning rather than starting from zero.

These are the project’s intended problem areas, not proven user-research findings.

CAPABILITY MAP

The platform combines a web interface with AI-assisted services

Intended or implemented-in-code capabilities, not verified end-to-end workflows.



New concept

Create a new concept from room dimensions, budget, and preferences.



Photo redesign

Analyze an uploaded bathroom photo and request a photo redesign.



Products and cost

Recommend products, calculate combined cost, and explore budget alternatives.



Design assistant

Offer design-edit suggestions through an assistant flow.

TECHNOLOGY STACK

Each technology has a different job

Technology	Role	Simple explanation
React	User interface	Builds the interactive frontend from reusable components
Vite	Frontend tooling	Runs the development server and builds frontend assets
Python + FastAPI	Backend API	Receives HTTP requests and returns responses
SQLAlchemy	Database access	Lets Python models work with a relational database
DATABASE_URL	Database selection	Chooses the configured database engine
Gemini, Stability AI, Pollinations AI	External services	Named in the README; exact workflow usage must be confirmed

SYSTEM ARCHITECTURE

System architecture: browser to backend to services

The request moves through the interface, API, Python processing, database access, and optional external AI services.













User selects room, style, budget, or uploads photo.



FRONTEND RESPONSIBILITIES

Frontend: collect inputs, call the API, show the result



React builds the interactive interface from reusable components.



The frontend is expected to collect room information, budget, style, and photo uploads.



It sends requests to the backend and displays returned results.



Vite supports fast local development and optimized frontend builds.



Known local frontend address: http://localhost:5173

BACKEND API

FastAPI routes the project's main workflows

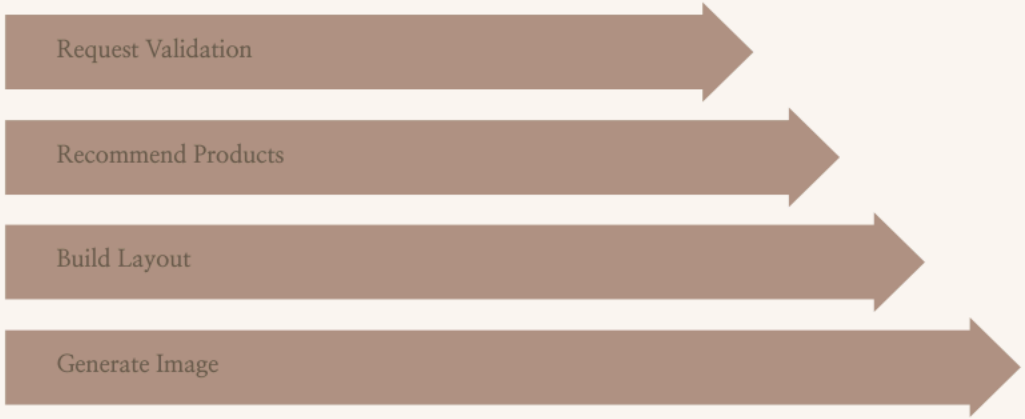
main.py is described as the backend entry point. FastAPI can expose interactive documentation at **/docs**.

Endpoint	Purpose
GET /	Returns a message indicating the backend is running
POST /analyze-bathroom	Passes image data to bathroom analysis functionality
POST /generate-photo-redesign	Requests a redesign of an uploaded bathroom photo
POST /generate-design	Validates inputs, recommends products, builds layout data, and calls image generation
POST /explore	Requests alternative budget options
POST /design-assistant	Requests a design-edit suggestion

MAIN DESIGN WORKFLOW

The main design workflow separates products, layout, and image output

1. Frontend sends room, style, and budget information.
2. FastAPI validates the request using DesignRequest.
3. recommend_products selects products; if none are available, the endpoint returns a failure response.
4. build_layout creates layout data from the room and selected products.
5. generate_design_image requests the image-generation step.
6. The response can include status, room, budget, theme, image filename, products, total product cost, remaining budget, and layout.



The response can include status, room, budget, theme, image filename, products, total product cost, remaining budget, and layout.

Keep layout data distinct from the generated image; successful end-to-end behavior depends on valid configuration and working external services.

DATABASE ACCESS LAYER

SQLAlchemy defines the database access layer

SQLAlchemy is an ORM, or Object-Relational Mapper: Python classes can represent relational database tables.

Table creation does not prove that every generated design is inserted and committed; persistence must be verified in the endpoint code.

